AIuthor — Prompts Dossier

Gateway Digital AI Engineer Assessment

Intent Analyzer

Purpose

Parse natural-language user requests into structured routing fields so LangGraph can branch to generate, tone-convert, insert, export, or repair without separate APIs.

Inputs

Field	Source
`user_input`	API body, Streamlit chat, or CLI

Outputs

Field	Type	Use
`task_type`	enum	Graph routing (`generate_book`, `tone_conversion`, `insert_chapter`, etc.)
`topic`, `reader`, `tone`, `genre`	strings	Brief and planner
`num_chapters`, `words_per_chapter`	int	Planner / loop bounds
`target_chapter`	int or null	Test C regenerate
`insert_after`, `source_run_id`	int/string or null	Test D insert
`character_names`	list	Fiction planner

Structured as `IntentResult` JSON (Pydantic).

Known failure modes

Failure	Mitigation
Ambiguous insert position (`insert_after` null or invalid)	`load_source` sets `needs_clarification`; graph ends until user supplies chapter (see dossier/10)
Wrong task type on vague input	Few-shot examples in prompt map assessment Tests A–D
Missing `source_run_id` for C/D	Heuristic regex + fallback to latest snapshot in `outputs/`
LLM unavailable / quota	Optional regex-only path when `MOCK_LLM=true` or `INTENT_SKIP_LLM_WHEN_HEURISTIC=true`

Why this prompt

• **Single front door**: Assessment requires one orchestration entry; intent must be machine-readable JSON, not prose.
• **Few-shot routing**: Examples tie assessment phrases (“10 chapter personal finance”, “regenerate chapter 3 academic”, “insert between 4 and 5”) to correct `task_type` without hard-coded parsers for

every phrasing.
• **Explicit null for insert**: Example with `insert_after: null` prevents silent default to chapter 4 (Test D integrity).
• **JSON-only**: Enables `invoke_structured` and deterministic graph edges without fragile regex on full books.

Full prompt

See [prompts/intent_analyzer.txt](../prompts/intent_analyzer.txt)

Planner

Purpose

Produce a publication-ready book outline—title, chapter list with summaries, and front/back matter plan—that seeds the chapter loop and assembler.

Inputs

Field	Source
`topic`, `reader`, `tone`, `genre`	Intent + brief
`num_chapters`, `words_per_chapter`	Intent
`character_names`	Intent (fiction)

Outputs

Field	Type	Use
`BookOutline`	JSON	Chapter loop, TOC, assembler
`title`, `subtitle`, `chapters[]`		Per-chapter `title`, `summary`, `target_words`
`front_matter_plan`, `back_matter_plan`	lists	Assembler surface checklist

Known failure modes

Failure	Mitigation
Weak chapter progression	Prompt requires logical progression and cross-chapter callbacks in summaries
Tone drift in titles	Tone repeated in prompt header; summaries must match tonality
Invalid JSON	`invoke_structured` + schema validation
Over-long outline for context	Planner runs once; summaries kept to 2–3 sentences

Why this prompt

• **Structured outline only**: JSON prevents regex scraping; downstream agents consume stable fields.

• **Assessment alignment**: `num_chapters` and `words_per_chapter` come from intent so Tests A/B hit requested scale.
• **Front/back matter plan**: Ensures assembler generates full book surfaces, not chapters-only PDFs.
• **Callback planning**: “Callbacks planned across chapters” in prompt seeds memory_keeper and writer continuity.

Full prompt

See [prompts/planner.txt](../prompts/planner.txt)

Researcher

Purpose

Ground non-fiction (and fact-heavy) chapters by extracting cited facts and glossary candidates from RAG-retrieved corpus chunks only.

Inputs

Field	Source
`topic`, `chapter_title`, `chapter_summary`, `tone`	Outline + brief
`retrieved_context`	ChromaDB + optional BM25 (`rag/retriever.py`)

Skipped when `skip_researcher_without_rag=true` and corpus is empty (facts list empty).

Outputs

Field	Type	Use
`ChapterResearch`	JSON	Writer, fact_checker, memory_keeper
`facts[]`	`{fact, source}`	Writer grounding; memory fact_registry
`references[]`	strings	Fact checker (generic only)
`glossary_candidates[]`	`{term, definition}`	Memory glossary

Known failure modes

Failure	Mitigation
Empty retrieval	Prompt: return fewer facts, do not hallucinate
Fabricated studies/ISBNs	“Every fact MUST cite source”; fact_checker second pass
LLM ignores corpus	Retrieved context injected in full; abstain rules in prompt
Duplicate facts across chapters	memory_keeper dedupes by chapter commit

Why this prompt

• **Citation-or-abstain**: Separates generation from evidence; supports assessment “grounded non-fiction” requirement.

• **JSON facts**: Writer receives structured list, not raw chunks—reduces token noise and contradiction.
• **Glossary at research time**: Terms introduced when chapter is researched, not invented at assembly.
• **Tone in research**: Definitions can match Academic vs Conversational before writer sees them.

Full prompt

See [prompts/researcher.txt](../prompts/researcher.txt)

Memory Keeper

Purpose

Persist cross-chapter continuity—facts, callbacks, glossary, character bible, tone fingerprint, decision log—and expose relevant slices to downstream prompts. Not an LLM agent; deterministic read/write around `BookMemory`.

Inputs

Phase	Fields
Read (`memory_read`)	`run_id`, `current_chapter`, existing `BookMemory`
Write (`memory_write`)	`research_by_chapter`, verified chapter text, `outline`, `chapters`, `brief`

Tone fingerprint loaded from tonality preset files (see dossier/14).

Outputs

Field	Use
Updated `BookMemory` on disk (`outputs/{run_id}/memory.json`)	Next chapter prompts
`snapshot.json`	Resume, tone_conversion, insert_chapter
Trace `memory_io.jsonl`	Observability

Known failure modes

Failure	Mitigation
Callback spam	Only first 120 chars of chapter stored as callback seed
Stale facts after insert	`repair.py` renumbers refs before write (after valid `insert_after`; see dossier/10)
Missing tone preset file	Defaults to generic rule list
Context bloat	`format_memory_context` caps callbacks/facts sent to prompts

Why this prompt (tonality presets, not LLM)

Memory Keeper has **no** `prompts/memory_keeper.txt`. Continuity is structural JSON, not generative. **Tonality presets** (`prompts/tonality/*.txt`) are the “prompt” for voice: bullet rules parsed into `ToneFingerprint` and injected as `{{tone_block}}` into writer, humanizer, combined/batch, and assembler—so tone cascades without re-asking the LLM what “Conversational” means each call.

Associated prompt files

See dossier/14_tonality_presets.md — [prompts/tonality/](../prompts/tonality/)

Implementation

`agents/memory_keeper.py` (no LLM invocation)

Writer

Purpose

Generate the first full draft of a chapter from outline, memory, grounded facts, and tone rules.

Inputs

Field	Source
`chapter_number`, `chapter_title`, `chapter_summary`, `target_words`, `genre`	Outline
`tone_block`	Memory `tone_fingerprint` (from tonality preset)
`memory_context`	Callbacks, prior facts, characters
`facts`	Researcher / RAG

Used in **split** pipeline mode only (`chapter_pipeline` → `split`).

Outputs

Field	Use
`raw_text`	Humanizer input

Known failure modes

Failure	Mitigation
Repetitive openings	Humanizer + optional preference-lite opening judge
Ungrounded non-fiction claims	Prompt: only use facts list; fact_checker later
AI tells in draft	Explicit banned-phrase list; humanizer enforces harder
Wrong tone	`tone_block` + tonality presets
Fiction character drift	Character bible in memory_context

Why this prompt

• **Draft-only contract**: Writer optimizes for coverage and structure; polish delegated to humanizer/editor—clear agent boundary for assessment.
• **Memory-aware**: Callbacks listed explicitly so chapters reference prior material (continuity requirement).
• **Genre split**: Non-fiction grounded vs fiction character rules in one template.
• **Prose-only output**: No JSON—reduces schema errors on long chapters.

Full prompt

See [prompts/writer.txt](../prompts/writer.txt)

Humanizer

Purpose

Rewrite draft prose to sound human: remove AI tells, vary rhythm, address the reader per tone, and weave memory callbacks.

Inputs

Field	Source
`chapter_text`	Writer `raw_text` (split mode)
`tone_block`	Memory / override
`memory_context`	Callbacks and facts

Outputs

Field	Use
`humanized_text`	Editor input

Known failure modes

Failure	Mitigation
Meaning drift	“Preserve meaning and factual content”
Wrong register (too casual/academic)	`tone_block` + REQUIRED humanization per tone
Banned phrases survive	Explicit MUST-remove list; paired with `ai_tell_eval` regex
Over-long output	Full chapter rewrite instructed

Known banned phrases (prompt-enforced)

• delve into · landscape of · in today's fast-paced world / in today's world
• it is important to note / it's important to note
• furthermore / moreover (stacked) · not only... but also (mechanical)
• leverage (buzzword) · robust / seamless / cutting-edge (empty modifiers)

Why this prompt

• **Dedicated anti-AI-tell agent**: Assessment scores human voice; separating humanization from writing avoids one prompt doing everything poorly.
• **Explicit ban list**: Models regress on “avoid AI language”; named phrases are auditable in dossier and evals.
• **Tone-specific behavior**: Second-person and metaphor rules differ for Conversational vs Academic in one place.
• **Callback pass**: Forces continuity at voice layer, not only fact layer.

Full prompt

See [prompts/humanizer.txt](../prompts/humanizer.txt)

Editor

Purpose

Polish chapter prose for readability, pacing, and tonal consistency without altering factual claims.

Inputs

Field	Source
`chapter_text`	Humanizer `humanized_text`
`book_title`, `chapter_number`	Outline / state

Outputs

Field	Use
`edited_text`	Fact checker input

Known failure modes

Failure	Mitigation
Fact alteration	“Do not change factual claims”
Over-cutting voice	Humanizer already set voice; editor focuses transitions/pacing
Generic smoothing	“Remove awkward phrasing” not “simplify to bland”

Why this prompt

• **Separation of concerns**: Humanizer owns voice; editor owns structure—mirrors editorial workflow in publishing.
• **Lightweight template**: Short prompt reduces tokens in split mode where five agents run per chapter.

• **Book-level consistency**: `book_title` + chapter number anchor cross-chapter tone.

Full prompt

See [prompts/editor.txt](../prompts/editor.txt)

Fact Checker

Purpose

Final chapter pass: verify claims against research, soften unverifiable statements, and block fabricated references.

Inputs

Field	Source
`chapter_text`	Editor `edited_text`
`verified_facts`	Researcher `facts`
`references`	Researcher `references`

Outputs

Field	Use
`verified_text`	memory_keeper commit; assembler chapter body

Known failure modes

Failure	Mitigation
Over-softening accurate claims	Only soften when cannot verify
Invented ISBNs/papers	NEVER invent; abstain rule
New facts added in check	“Do not add new factual claims”
Empty research	Softens or removes bold claims

Why this prompt

• **Last line of defense**: Writer/humanizer may drift; checker aligned to research artifacts only.
• **Abstain over fabricate**: Matches researcher citation discipline for assessment trust.
• **Prose output**: Same contract as editor/humanizer for pipeline chaining.

Full prompt

See [prompts/fact_checker.txt](../prompts/fact_checker.txt)

Assembler

Purpose

Generate all front and back matter in the book’s tone, merge with chapter bodies, and export PDF/DOCX plus `manifest.json`.

Inputs

Field	Source
`title`, `topic`, `reader`	Outline / brief
`tone_block`	Memory tone fingerprint
`chapter_titles`	Completed chapters
`glossary_preview`	Memory glossary terms

Outputs

Field	Use
`BookManifest` JSON	`file_generator` → PDF, DOCX
Half-title, copyright, dedication, epigraph, foreword, preface, acknowledgments, introduction, afterword, appendix, glossary, references, about_author, back_cover_copy	Publication surfaces

Known failure modes

Failure	Mitigation
Generic back matter ignoring tone	“Tone MUST cascade to every surface”
Invented real ISBNs	Placeholder `978-0-000000-00-0` in prompt example only
JSON schema drift	`invoke_structured` + manifest schema
Glossary mismatch	`glossary_preview` from memory

Why this prompt

• **Tonality beyond chapters**: Assessment requires tone on preface, glossary, back cover—not just chapter bodies.
• **Structured manifest**: One JSON blob drives deterministic PDF/DOCX layout.
• **Explicit surface list**: Mirrors `front_matter_plan` / `back_matter_plan` from planner.
• **Glossary in voice**: Definitions must match tone (Conversational friend vs Academic precision).

Full prompt

See [prompts/assembler.txt](../prompts/assembler.txt)

Insert Chapter Clarification

Purpose

Ensure Test D–style inserts never guess `insert_after=4` when the user omits or invalidates insert position. **No LLM prompt**—deterministic parsing and validation in code.

Inputs

Field	Source
User chat reply or API `insert_after`	Streamlit / `POST /execute`
`pending_insert`	Prior `needs_clarification` response
`source_run_id`, chapter count	Snapshot

Outputs

Field	Use
Valid `insert_after` ($1 \le n < \text{total\_chapters}$)	`prepare_insert` → repair → chapter pipeline
`needs_clarification` + `clarification_message`	END until resolved

Flow

1. Intent Analyzer sets `insert_chapter` and `insert_after` (integer or null).
2. `load_source` validates range.
3. If invalid → `status: needs_clarification`, graph END.
4. User supplies position; workflow resumes with preset `insert_after` / `source_run_id` (intent pass-through).

Accepted user replies (regex)

• `4` · `after chapter 4` · `between chapter 4 and 5`

Known failure modes

Failure	Mitigation
Silent default to chapter 4	No default in code; clarification required
Insert at last chapter	Validation rejects; user must pick valid gap
Stale chat thread	Streamlit **New chat** = new `run_id` / trace folder

Why no prompt

Clarification is **structured user input**, not generation. Regex + validation are faster, cheaper, and auditable than an LLM guessing chapter numbers.

Implementation

• [agents/insert_clarification.py](../agents/insert_clarification.py)

• [graph/nodes.py](../graph/nodes.py) (`node_load_source`, intent pass-through)
• [ui/streamlit_app.py](../ui/streamlit_app.py) · [api/main.py](../api/main.py)

Chapter Combined (adaptive pipeline)

Purpose

Produce **final chapter prose in one LLM call** by fusing research, write, humanize, edit, and fact-check instructions—used when `chapter_pipeline_mode=auto` selects **combined** (default per-chapter budget).

Inputs

Field	Source
`chapter_number`, `chapter_title`, `chapter_summary`, `target_words`, `genre`	Outline
`tone_block`	Memory / `tone_override` (Test C)
`memory_context`	memory_read
`facts`	RAG retrieval + researcher-style fact list (built in pipeline)

Outputs

Field	Use
Final chapter prose (all text fields set to same body)	memory_write → assembler

Agent trace name: `chapter_combined`. Replaces split chain: researcher → writer → humanizer → editor → fact_checker for that chapter.

Known failure modes

Failure	Mitigation
Skipped explicit humanizer pass	Prompt lists all five internal steps + banned AI tells
Weaker fact-check vs split	Corpus + “do not invent statistics/ISBNs” in prompt
Context overflow	`choose_chapter_pipeline_mode` falls back to **split**
Tone override ignored	`tone_override` injected into `tone_block`

Why this prompt

• **Token and latency budget**: Assessment-scale books need fewer calls; one strong call beats five cheap ones when context fits.
• **Explicit multi-step instruction**: Models behave better when told to research→write→humanize→edit→verify internally.
• **Same grounding rules as split agents**: Non-fiction abstain rules preserved in one template.
• **Prose-only output**: Avoids JSON truncation on long chapters (unlike batch mode).

Full prompt

See [prompts/chapter_combined.txt](../prompts/chapter_combined.txt)

Chapters Batch (adaptive pipeline)

Purpose

Write **all chapters in one structured LLM call** when the job is small enough (few chapters, short word targets)—`chapter_pipeline_mode=auto` selects **batch**.

Inputs

Field	Source
`title`, `topic`, `genre`	Outline / brief
`tone_block`	Memory
`memory_context`	memory_read (chapter 1 context for whole book)
`chapter_specs`	Formatted list: number, title, summary, target words per chapter
`facts`	Aggregated RAG excerpts per chapter

Outputs

Field	Use
`ChaptersBatchOutput` JSON `{ chapters: [{chapter_number, title, text}] }`	All chapters updated; per-chapter memory_write

Known failure modes

Failure	Mitigation
JSON truncation on long books	`batch_chapters_max` + context budget → use combined/split instead
Weaker per-chapter memory	Batch uses chapter-1 memory slice; acceptable for short POC runs
Inconsistent voice across chapters	Shared `tone_block` and single call
Partial chapter failure	Structured output validation; failed parse retries via LLM client

Why this prompt

• **Minimum calls for demo/short books**: 1 LLM call for N chapters when N×words fits context.
• **JSON required**: Multiple chapters must return machine-parseable array; unlike combined single-chapter prose mode.
• **Same fused pipeline as combined**: Each chapter still instructed to research→write→humanize→edit→fact-check.
• **Paired with auto mode**: `utils/context_budget.py` only selects batch when safe.

Full prompt

See [prompts/chapters_batch.txt](../prompts/chapters_batch.txt)

Tone Eval (evaluation judge)

Purpose

Score how well a text sample matches the requested tone (1–5). Used by `evals/tone_eval.py` and optional **preference-lite** opening selection (`pick_best_opening`).

Inputs

Field	Source
`tone`	Brief / Test C override (Conversational, Academic, etc.)
`text_sample`	First ~4000 chars of chapter or opening variant

Outputs

Field	Use
Numeric score 1–5 + one-sentence justification	Normalized to 0–1; `passed` if ≥ 0.6
Eval report	`evals/run_evals.py` when `auto_run_evals=true`

Not part of the LangGraph book pipeline; runs post-hoc or inside humanizer variant loop.

Known failure modes

Failure	Mitigation
Non-numeric LLM reply	Regex extract first number; default 0.7 on parse failure
Judge too lenient	Paired with `ai_tell_eval` banned-phrase checks
Short sample	Empty text → score 0, failed

Why this prompt

• **Measurable tonality**: Assessment asks for tone fidelity; a dedicated judge is reproducible in eval reports.
• **Minimal output**: Single number + sentence keeps judge calls cheap (`tier=cheap`).
• **Preference-lite path**: Documents route to full DPO/RLHF without training infra—compare opening variants by score.
• **Test C support**: Validates regenerate-in-new-tone actually shifted voice.

Full prompt

See [prompts/eval_tone.txt](../prompts/eval_tone.txt)

Tonality Presets (shared prompt fragments)

Purpose

Define **tone rules** loaded by Memory Keeper into `ToneFingerprint`, then injected as `{{tone_block}}` into Writer, Humanizer, Chapter Combined, Chapters Batch, and Assembler. Not standalone LLM calls—fragments concatenated into agent prompts.

Inputs

Field	Source
`tone` name from intent	`Conversational`, `Academic`, `Storyteller`, `Motivational`, `Witty`
File path	`prompts/tonality/{tone.lower()}.txt`

Outputs

Field	Use
`ToneFingerprint { tone, rules[] }`	`format_tone_block()` in agent prompts

Presets (one file per assessment tone)

Conversational
• **File**: [prompts/tonality/conversational.txt](../prompts/tonality/conversational.txt)
• **Why**: Test A default; second-person, simple language, friendly glossary.
• **Failure modes**: Drift to academic jargon → rules forbid formal structure.

Academic
• **File**: [prompts/tonality/academic.txt](../prompts/tonality/academic.txt)
• **Why**: Test C regenerate target; third-person, evidence framing, scholarly glossary.
• **Failure modes**: Dry lists → writer/humanizer still require hooks where prompt allows.

Storyteller
• **File**: [prompts/tonality/storyteller.txt](../prompts/tonality/storyteller.txt)
• **Why**: Test B novella; scene, pacing, in-world glossary.
• **Failure modes**: Over-exposition → show-through-action rules.

Motivational
• **File**: [prompts/tonality/motivational.txt](../prompts/tonality/motivational.txt)
• **Why**: Test C option; direct address, action steps, energizing back cover.
• **Failure modes**: Empty hype → “anchor encouragement in specifics”.

Witty
• **File**: [prompts/tonality/witty.txt](../prompts/tonality/witty.txt)
• **Why**: Test C option; humor without undermining trust.
• **Failure modes**: Sarcasm eroding credibility → explicit avoid rule.

Known failure modes (all presets)

Failure	Mitigation
Missing file for tone name	Fallback: single generic rule in `memory_keeper._load_tone_preset`
Tone only in chapters, not matter	Assembler prompt requires cascade to all surfaces
Override not applied	Test C sets `tone_override` on state before chapter pipeline

Why separate preset files (not inline in writer.txt)

• **Single source of truth**: Change Conversational once; all agents inherit via `tone_block`.
• **Assessment tones map 1:1** to files for eval and documentation.
• **Assembler + chapter agents stay DRY**: Same fingerprint for body and back matter.
• **Eval alignment**: `eval_tone.txt` judges against the same tone label the preset defines.

Full prompt files

Tone	Path
Conversational	[prompts/tonality/conversational.txt](../prompts/tonality/conversational.txt)
Academic	[prompts/tonality/academic.txt](../prompts/tonality/academic.txt)
Storyteller	[prompts/tonality/storyteller.txt](../prompts/tonality/storyteller.txt)
Motivational	[prompts/tonality/motivational.txt](../prompts/tonality/motivational.txt)
Witty	[prompts/tonality/witty.txt](../prompts/tonality/witty.txt)

Appendix: Full Prompt Files

prompts\assembler.txt

```
You are the Assembler agent. Generate all front and back matter in the book's tone.
Title: {{title}}
Topic: {{topic}}
Reader: {{reader}}
{{tone_block}}
Chapter list:
{{chapter_titles}}
Glossary preview:
{{glossary_preview}}
Tone MUST cascade to every surface — preface, afterword, glossary-style definitions,
about-the-author, and back-cover copy must all match the tone (not generic).
Return JSON only:
{
"title": "{{title}}",
"half_title": "...",
"copyright_block": "full copyright with ISBN placeholder 978-0-000000-00-0, edition,
rights, CIP block",
"dedication": "...",
"epigraph": "...",
"foreword": "...",
"preface": "...",
"acknowledgments": "...",
"introduction": "...",
"afterword": "...",
"appendix": "...",
"glossary": {},
"references": [],
"about_author": "...",
```

```
"back_cover_copy": "..."
}
```

prompts\chapter_combined.txt

```
You are the chapter production agent for AIuthor. In ONE pass, produce the final chapter
prose.
Do all of the following internally before you write:
1. Research — use grounded facts and corpus excerpts only; do not invent statistics or
ISBNs.
2. Write — engaging draft at target length for the genre.
3. Humanize — natural voice per tone rules; no AI tells ("delve into", "landscape of",
"it's important to note").
4. Edit — pacing, transitions, consistency with memory.
5. Fact-check — soften or remove unverifiable claims.
Chapter {{chapter_number}}: {{chapter_title}}
Summary: {{chapter_summary}}
Target length: ~{{target_words}} words
Genre: {{genre}}
{{tone_block}}
Memory context:
{{memory_context}}
Corpus / facts:
{{facts}}
Output ONLY the final chapter prose (no JSON, no step labels).
```

prompts\chapters_batch.txt

```
You are the chapter production agent for AIuthor. In ONE pass, write ALL chapters below.
For each chapter: research from corpus, write, humanize, edit, and fact-check — then
output final prose only.
Book: {{title}}
Topic: {{topic}}
Genre: {{genre}}
{{tone_block}}
Memory context:
{{memory_context}}
Chapters to write:
{{chapter_specs}}
Corpus excerpts:
{{facts}}
Return JSON only:
{
"chapters": [
{"chapter_number": 1, "title": "...", "text": "full chapter prose..."}
]
}
```

prompts\editor.txt

```
You are the Editor agent. Polish the chapter for readability, pacing, and consistency.
Book: {{book_title}}
Chapter: {{chapter_number}}
Improve:
- Transitions between sections
- Paragraph pacing
- Consistency with prior tone
- Remove any remaining awkward phrasing
Do not change factual claims. Output the full edited chapter only.
Chapter text:
{{chapter_text}}
```

prompts\eval_tone.txt

```
Rate how strongly the following text matches the {{tone}} tone on a scale of 1-5.
1 = completely wrong tone
5 = perfect match
Text:
{{text_sample}}
Respond with a single number 1-5 and one sentence justification.
```

prompts\fact_checker.txt

```
You are the Fact Checker agent. Verify factual claims against provided sources.
Verified facts from research:
{{verified_facts}}
References:
{{references}}
Rules:
- If a claim cannot be verified, soften language ("many experts suggest") or remove it
- NEVER invent references to real books, papers, or ISBNs
- Do not add new factual claims
- Abstain rather than fabricate certainty
Output the verified chapter text only.
Chapter to verify:
{{chapter_text}}
```

prompts\humanizer.txt

```
You are the Humanizer agent. Rewrite the chapter so it sounds deeply human and talks TO
the reader.
{{tone_block}}
Memory context (use callbacks):
{{memory_context}}
BANNED PHRASES — remove or replace every occurrence:
- delve into
- landscape of
- in today's fast-paced world / in today's world
- it is important to note / it's important to note
- furthermore / moreover (when stacked mechanically)
- not only... but also (symmetric mechanical pairs)
- leverage (as buzzword)
- robust / seamless / cutting-edge (empty modifiers)
REQUIRED humanization:
- Varied sentence lengths (short punchy lines mixed with longer flowing ones)
- Emotional hooks aligned with tone
- Second-person address where tonality supports it (especially Conversational,
Motivational)
- Domain-drawn metaphors (not generic business metaphors)
- Memory-backed callbacks referenced naturally
- Storytelling rhythm — no textbook enumeration unless Academic tone
Preserve meaning and factual content. Output the full rewritten chapter only.
Original chapter:
{{chapter_text}}
```

prompts\intent_analyzer.txt

```
You are the Intent Analyzer for AIuthor. Parse the user's natural-language request into
a structured task.
User input:
{{user_input}}
Return JSON only with this schema:
{
"task_type": "generate_book|tone_conversion|insert_chapter|export_book|repair_book|rege
nerate_chapter",
"topic": "string",
```

```
"reader": "string",
"tone": "Conversational|Academic|Storyteller|Motivational|Witty",
"genre": "non-fiction|fiction",
"num_chapters": 10,
"words_per_chapter": 2500,
"target_chapter": null or integer,
"insert_after": null or integer,
"source_run_id": null or string,
"character_names": [],
"length": ""
}
Examples:
- "10 chapter beginner guide to personal finance conversational" -> generate_book, topic
Personal Finance, 10 chapters
- "regenerate chapter 3 academic" -> tone_conversion, target_chapter 3, tone Academic
- "insert chapter between 4 and 5" -> insert_chapter, insert_after 4
- "insert a chapter in book run abc" (no position) -> insert_chapter, insert_after null
```

prompts\planner.txt

```
You are an expert book planner. Create a compelling, publication-ready outline.
Topic: {{topic}}
Reader: {{reader}}
Tone: {{tone}}
Genre: {{genre}}
Number of chapters: {{num_chapters}}
Target words per chapter: {{words_per_chapter}}
Named characters (fiction): {{character_names}}
Return JSON only:
{
"title": "compelling book title",
"subtitle": "optional subtitle",
"genre": "{{genre}}",
"chapters": [
{
"chapter_number": 1,
"title": "chapter title",
"summary": "2-3 sentence summary with pacing notes",
"target_words": {{words_per_chapter}}
}
],
"front_matter_plan": ["half-title", "title", "copyright", "dedication", "epigraph",
"toc", "foreword", "preface", "acknowledgments", "introduction"],
"back_matter_plan": ["afterword", "appendix", "glossary", "references",
"about-the-author", "back-cover"]
}
Ensure logical progression, callbacks planned across chapters, and tone-appropriate
chapter titles.
```

prompts\researcher.txt

```
You are the Researcher agent. Extract grounded facts from retrieved context ONLY.
Topic: {{topic}}
Chapter: {{chapter_title}}
Summary: {{chapter_summary}}
Tone: {{tone}}
Retrieved context:
{{retrieved_context}}
Rules:
- Every fact MUST cite its source from retrieved context
- Do NOT invent statistics, studies, or book titles
- If context is insufficient, return fewer facts rather than hallucinate
Return JSON only:
{
"chapter_number": 0,
"facts": [{"fact": "...", "source": "doc name or chunk"}],
"references": ["safe generic references only — no fabricated real ISBNs"],
```

```
"glossary_candidates": [{"term": "...", "definition": "tone-appropriate definition"}]
}
```

prompts\tonality\academic.txt

```
Tone: Academic
- Use precise terminology with clear definitions
- Structured arguments with evidence-based claims
- Third-person preferred; formal register
- Avoid colloquialisms and rhetorical questions
- Glossary definitions should be scholarly and precise
- Preface and afterword use analytical framing
```

prompts\tonality\conversational.txt

```
Tone: Conversational
- Speak directly to the reader using "you"
- Use simple, everyday language
- Short paragraphs and friendly asides
- Avoid academic jargon and formal structure
- Use questions to engage the reader
- Glossary definitions should sound like a friend explaining, not a textbook
```

prompts\tonality\motivational.txt

```
Tone: Motivational
- Direct address to inspire action
- Affirm reader capability; concrete next steps
- Energetic rhythm; triumph over obstacle framing
- Avoid empty hype — anchor encouragement in specifics
- Glossary definitions empower the reader
- Back-cover copy drives urgency to begin
```

prompts\tonality\storyteller.txt

```
Tone: Storyteller
- Narrative-driven prose with scene and sensory detail
- Show character through action and dialogue
- Vary pacing — tension, release, reflection
- Metaphors drawn from the story world
- Glossary as in-world terms or narrator asides
- Foreword/preface feel like an invitation into a tale
```

prompts\tonality\witty.txt

```
Tone: Witty
- Clever observations without forced jokes
- Light irony and unexpected turns of phrase
- Still substantive — humor serves clarity
- Avoid sarcasm that undermines trust
- Glossary definitions may include playful asides
- About-the-author can be self-deprecating charm
```

prompts\writer.txt

```
You are the Writer agent. Write an engaging, publication-quality chapter.
Chapter {{chapter_number}}: {{chapter_title}}
```

```
Summary: {{chapter_summary}}
Target length: ~{{target_words}} words
Genre: {{genre}}
{{tone_block}}
Memory context:
{{memory_context}}
Grounded facts (use for non-fiction; do not contradict):
{{facts}}
Requirements:
- Natural prose with varied sentence rhythm
- Open with a tonality-appropriate hook
- Weave in callbacks from memory where relevant
- For non-fiction: only state facts supported by the facts list
- For fiction: maintain character consistency from memory
- Avoid AI tells: no "delve into", "landscape of", "it's important to note", mechanical
triads
Write the full chapter prose only (no JSON).
```